МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта

Кафедра проблем управления

ПРАКТИЧЕСКАЯ РАБОТА №1

по дисциплине: **Информационные элементы робототехнических систем**

Тема лабораторной работы: «Обработка изображений: шумоподавление и гамма-коррекция»

Студенты группы: КРБО-03-23 Зенина А.А. _____

Грачев А.В. _____

Преподаватель: ст. преподаватель Бычков А.М. _____

Работа представлена к защите: «___» _____ 2026 г.

Москва 2026

1. Цель работы

Изучить методы цифровой обработки изображений: преобразование в градации серого, наложение шума «соль-перец», применение сглаживающих фильтров (усредняющего, Гаусса, медианного), гамма-коррекцию; освоить построение гистограмм для анализа яркостных характеристик.

2. Задача работы

1. Реализовать на языке Python функции для перевода цветного изображения в оттенки серого, добавления шума «соль-перец».
2. Применить фильтры: усредняющий (box blur) с ядрами 3×3, 5×5, 7×7; фильтр Гаусса с ядром 3×3; медианный фильтр с ядром 3×3.
3. Выполнить гамма-коррекцию исходного цветного изображения с коэффициентом $\gamma = 0.3$.
4. Построить гистограммы исходного изображения и после гамма-коррекции.
5. Проанализировать полученные результаты и сделать выводы.

3. Теоретические сведения

Цифровая обработка изображений включает множество операций, направленных на улучшение визуального качества или подготовку к автоматическому анализу.

- Шум «соль-перец» (импульсный шум) проявляется в виде случайных белых и чёрных пикселей. Возникает из-за ошибок передачи данных или сбоёв сенсора.
- Усредняющий фильтр (box blur) заменяет каждый пиксель средним арифметическим яркостей в прямоугольной окрестности. Прост в реализации, но сильно размывает границы.
- Фильтр Гаусса использует свёртку с ядром, коэффициенты которого вычислены по функции Гаусса. Обеспечивает более естественное размытие, лучше сохраняет границы по сравнению с прямоугольным усреднением.
- Медианный фильтр заменяет центральный пиксель медианой значений в окне. Превосходно удаляет импульсный шум, почти не размывая резкие перепады яркости.
- Гамма-коррекция — нелинейное преобразование яркости $V_{out} = V_{in}^{gamma}$. При gamma < 1 тёмные области осветляются, при gamma > 1 — затемняются. Позволяет скорректировать изображение, снятое с неправильной экспозицией, или подготовить его для вывода на устройство с нелинейной характеристикой.

- Гистограмма изображения показывает распределение пикселей по яркости. По её форме можно судить о контрасте, засветке или затемнённости кадра.

4. Расчетно-графическая часть

4.1. Исходное изображение и его гистограмма

На рисунке 1 представлено исходное цветное изображение. Его гистограмма (рисунок 2) показывает, что пики яркости сильно смещены влево – изображение затемнено.

Рисунок 1 – Исходное цветное изображение

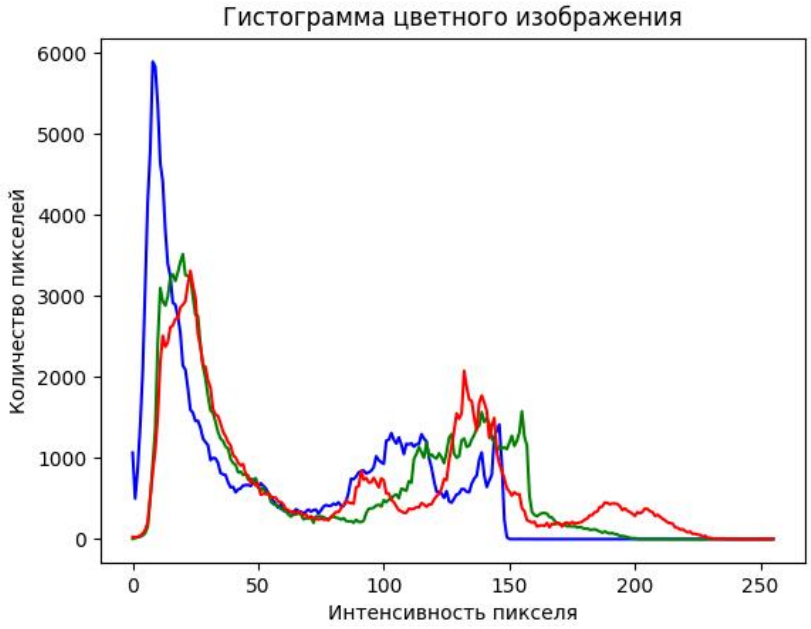

Рисунок 2 – Гистограмма исходного изображения

4.2. Преобразование в градации серого

Для упрощения дальнейшей обработки изображение переведено в оттенки серого (рисунок 3). Использовано взвешенное усреднение каналов: Y = 0.299 R + 0.587 G + 0.114 B.

Рисунок 3 – Изображение в градациях серого

4.3. Наложение шума «соль-перец»

На полученное полутоновое изображение наложен импульсный шум с вероятностью 0.05 (рисунок 4). Белые и чёрные точки имитируют помехи канала передачи.

Рисунок 4 – Изображение с шумом «соль-перец» (p=0.05)

4.4. Фильтрация зашумлённого изображения

Для подавления шума применены три типа фильтров.

Усредняющий фильтр (размер ядра 3×3, 5×5, 7×7) – рисунки 5, 6, 7 соответственно.

Фильтр Гаусса (ядро 3×3, σ = 1.0) – рисунок 8.

Медианный фильтр (ядро 3×3) – рисунок 9.

Рисунок 5 – Усредняющий фильтр 3×3

Рисунок 6 – Усредняющий фильтр 5×5

Рисунок 7 – Усредняющий фильтр 7×7

Рисунок 8 – Фильтр Гаусса 3×3 (σ=1.0)

Рисунок 9 – Медианный фильтр 3×3

Визуальный анализ показывает, что медианный фильтр наилучшим образом удаляет шум, сохраняя при этом чёткость контуров. Усредняющий фильтр оставляет заметные «разводы» и сильно размывает изображение. Фильтр Гаусса занимает промежуточное положение.

4.5. Гамма-коррекция исходного изображения

К исходному цветному изображению применена гамма-коррекция с параметром $\gamma = 0.3$. Результат показан на рисунке 10, а его гистограмма – на рисунке 11. Видно, что после коррекции гистограмма растянулась вправо, изображение стало светлее, проявились детали в тенях.

Рисунок 10 – Гамма-коррекция γ=0.3

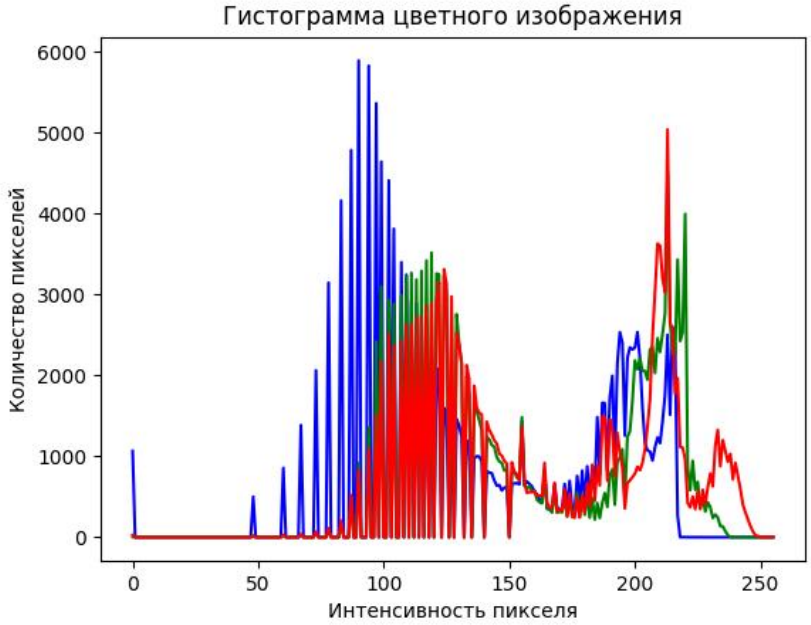

Рисунок 11 – Гистограмма после гамма-коррекции

Таким образом, гамма-коррекция эффективно компенсирует недостаточную экспозицию.

5. Выводы по работе

В ходе выполнения практической работы были изучены и программно реализованы базовые методы обработки изображений: преобразование в градации серого, наложение

импульсного шума, фильтрация (среднее арифметическое, Гаусса, медианная фильтрация), гамма-коррекция, построение гистограмм.

Экспериментально подтверждено, что:

- медианный фильтр наиболее эффективен для удаления шума «соль-перец»;
- гамма-коррекция с $\gamma < 1$ позволяет осветлить недоэкспонированные изображения и улучшить их визуальное восприятие;
- гистограммы служат удобным инструментом для анализа яркостных характеристик и контроля результатов коррекции.

Цель работы достигнута, все запланированные задачи выполнены.

ПРИЛОЖЕНИЕ А

Листинг программы, реализующей все описанные алгоритмы.

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

def show_image(img, name):
    if img is not None:
        cv2.imshow(f'{name}', img)          # показать окно
        cv2.waitKey(0)                     # ждать нажатия клавиши
        cv2.destroyAllWindows()            # закрыть окна
    else:
        print('Не удалось загрузить изображение')

def draw_histogram(img):
    # Для цветного изображения - гистограмма по каналам
    colors = ('b', 'g', 'r')
    for i, color in enumerate(colors):
        hist = cv2.calcHist([img], [i], None, [256], [0, 256])
        plt.plot(hist, color=color)

    plt.title('Гистограмма цветного изображения')
    plt.xlabel('Интенсивность пикселя')
    plt.ylabel('Количество пикселей')
    plt.show()

def get_grasclae_img(img):

    height, width, channels = img.shape

    # Создаем пустую матрицу для grayscale
    gray = np.zeros((height, width), dtype=np.uint8)

    # Попиксельное преобразование
    for i in range(height):
        for j in range(width):
            # Получаем значения BGR
            b, g, r = img[i, j]

            # Преобразование в grayscale (взвешенное среднее)
            gray_value = int(0.114  b + 0.587  g + 0.299  r)
            gray[i, j] = gray_value

    return gray

def add_salt_pepper_noise(img, prob):
    """
    Добавляет шум "соль-перец" к изображению.
    prob - вероятность замены пикселя шумом (0..1).
    """
    noisy = np.copy(img)
    # Генерируем маску для каждого пикселя
    random_mask = np.random.random(img.shape[:2])
    # Соль (белый) для всех каналов
    noisy[random_mask < prob/2] = 255
    # Перец (чёрный) для всех каналов
    noisy[random_mask > 1 - prob/2] = 0
    return noisy
```

```
def add_blurr(img, kernel_size):
    height, width = img.shape
    blurred = np.zeros_like(img, dtype=np.float32)
    pad = kernel_size // 2
    padded_img = np.pad(img, pad, mode='constant', constant_values=0)

    for i in range(height):
        for j in range(width):
            neighborhood = padded_img[i:i+kernel_size, j:j+kernel_size]
            blurred[i, j] = np.mean(neighborhood)

    return blurred.astype(np.uint8)

def add_gaussian(img, kernel_size, sigma=1.0):
    kernel = np.zeros((kernel_size, kernel_size))
    center = kernel_size // 2
    pad = kernel_size // 2

    for i in range(kernel_size):
        for j in range(kernel_size):
            x = i - center  # смещение по вертикали от центра
            y = j - center  # смещение по горизонтали от центра
            kernel[i, j] = np.exp(-(x2 + y2) / (2  sigma2))

    kernel = kernel / np.sum(kernel)

    height, width = img.shape
    gaussian = np.zeros_like(img, dtype=np.float32)
    padded_img = np.pad(img, pad, mode='constant', constant_values=0)

    for i in range(height):
        for j in range(width):
            neighborhood = padded_img[i:i+kernel_size, j:j+kernel_size]
            gaussian[i, j] = np.sum(neighborhood  kernel)

    return gaussian.astype(np.uint8)

def add_median(img, kernel_size):
    height, width = img.shape
    median = np.zeros_like(img)

    pad = kernel_size // 2
    padded_img = np.pad(img, pad, mode='constant', constant_values=0)

    for i in range(height):
        for j in range(width):
            # Извлекаем окрестность пикселя
            neighborhood = padded_img[i:i+kernel_size, j:j+kernel_size]
            # Берем медиану (средний элемент в отсортированном списке)
            median[i, j] = np.median(neighborhood)

    return median

def gamma_correction(img, gamma=0.5):
    height, width, channels = img.shape
    result = np.zeros_like(img)
    for i in range(height):          # проходим по всем строкам
        for j in range(width):       # проходим по всем столбцам
            for c in range(channels): # проходим по всем цветам
                pixel_value = img[i, j, c]
```

```
                normalized = pixel_value / 255.0
                corrected = normalized  gamma
                new_value = corrected  255
                result[i, j, c] = np.clip(round(new_value), 0, 255)

    return result.astype(np.uint8)

def main():

    img = cv2.imread('orig.jpg')

    # 1. Вывести оригинал + гистограмма
    show_image(img, 'Original')
    draw_histogram(img)

    # 2. Вывести grayscale
    gray = get_grasclae_img(img)
    cv2.imwrite('grayscale.jpg', gray)
    show_image(gray, 'Grayscale')

    # 3. Добавить шум "соль-перец"
    noisy = add_salt_pepper_noise(gray, 0.05)
    cv2.imwrite('noisy-salt-pepper.jpg', noisy)
    show_image(noisy, 'Noisy-salt-pepper')

    for i in range (3, 10, 2):
        blurr = add_blurr(noisy, i)
        cv2.imread(f'blurred_{i}.jpg', blurr)
        show_image(blurr, f'Blurred_{i}')

    gaussian = add_gaussian(noisy, 3)
    cv2.imwrite('gaussian.jpg', gaussian)
    show_image(gaussian, 'Gaussian')


    median = add_median(noisy, 3)
    cv2.imwrite('median.jpg', median)
    show_image(median, 'Median')

    gamma = gamma_correction(img, 0.3)
    cv2.imwrite('gamma.jpg', gamma)
    show_image(gamma, 'Gamma')

    draw_histogram(gamma)

    cv2.waitKey(1)  # небольшой дополнительный WAIT для обработки событий
    cv2.destroyAllWindows()
    cv2.waitKey(1)  # ещё один, чтобы гарантировать закрытие


if __name__ == '__main__':
    main()
```